Cross-Lingual Constraint Compliance Under Sequential Delegation in AI Agent Chains

Anonymous

March 18, 2026

Exploratory study · 252 trials across 2 runs · $29.24 total cost

Abstract. When AI coding agents delegate tasks through manager–worker chains, safety constraints specified in the manager's instructions must survive the delegation handoff. Prior work has established high constraint fidelity (99.4%) in English-language chains. We investigate what happens when the *entire* agent configuration is expressed in a non-English language (Italian or Russian). Across 252 trials in two experimental runs using Claude Code's native Agent tool with Claude Sonnet 4.6, we report three findings. First, the manager's delegation language is sensitive to the *trigger prompt*: when triggered in English ("Follow the instructions"), 100% of delegations are in English; when triggered in the target language, 26% of Italian and 14% of Russian delegations preserve the original language. Second, constraint fidelity remains near-perfect ($\geq$98%) regardless of language or trigger. Third, when compliance is measured proportionally (fraction of compliant symbols rather than binary pass/fail), apparent cross-lingual variation collapses: the everything-oneliner constraint achieves $\approx$97.5% compliance uniformly across languages, revealing that binary scoring in Run 1 inflated small per-trial differences into large aggregate gaps. The dominant predictor of compliance remains constraint–task interaction (framework conventions), not input language.

1 Introduction

As large language model (LLM) agents are deployed in increasingly complex software engineering workflows, a common architectural pattern has emerged: *sequential delegation*, where a manager agent decomposes a task and delegates subtasks to worker agents. This pattern is native to tools like Claude Code, which provides an Agent tool for spawning sub-agents within a coding session.

A critical concern in such chains is *constraint survival*: when a user specifies safety or style constraints (e.g., "do not use global variables"), do those constraints propagate faithfully through the delegation handoff? Air [1] demonstrated that in English-language Claude Code chains, constraint fidelity is remarkably high (99.4%), though compliance—whether the worker's code actually satisfies the constraint—lags at 94.3%, with constraint polarity as the dominant predictor.

However, software development is a global activity. Developers may naturally express constraints in their own language. What happens when the entire agent configuration is non-English? We designed a full-immersion experiment to test this, running two experimental runs that revealed three unexpected findings:

1. The manager's delegation language is not fixed—it is influenced by the language of the *trigger prompt* (the `-p` argument to the CLI), creating a previously unreported confound in the experimental design itself.

2. Constraint fidelity is robust across languages (≥98%), even when the manager preserves the non-English language in its delegation.
3. Binary compliance scoring (pass/fail per trial) inflates small per-function differences into large aggregate effects. When measured proportionally, cross-lingual compliance variation disappears.

2 Related Work

2.1 Value Drift in Agent Chains

Air [1] conducted the most comprehensive study of constraint propagation in LLM agent chains to date. Using Claude Code's native Agent tool with 13 coding constraints, 5 Python tasks, 2 models (Opus and Sonnet 4.6), and 2 prompting conditions across 600 trials, they found that fidelity is effectively solved at 99.4%. Compliance averaged 94.3%, with constraint polarity as the dominant predictor: positive constraints ("do X") achieved 90.8% compliance versus 98.0% for negative constraints ("don't do X").

Our study extends this line of inquiry to the cross-lingual setting. We use a similar constraint taxonomy and scoring approach, inducing the same manager→worker chain through Claude Code's Agent tool. Our findings about fidelity (≥98%) are consistent with Air's (99.4%), though direct numerical comparison requires caution given differences in constraint selection and trial count.

2.2 Multilingual LLM Capabilities

Evaluation benchmarks consistently show LLM performance degradation on non-English tasks [2], with the gap widening for lower-resource languages. In the coding domain, multilingual code generation benchmarks [3] show that prompting in English generally yields better code, though the gap has narrowed with newer models. Our study differs in that we do not measure raw code quality—the model generates functional code in all conditions—but whether *auxiliary style constraints* survive the delegation chain when expressed in non-English languages.

2.3 Principal-Agent Theory and AI Delegation

The manager–worker pattern maps onto classical principal–agent theory [4], where information asymmetry between principal and agent can lead to goal misalignment. The manager's decision to translate constraints to English—rather than passing them through verbatim—is an example of editorial discretion over the communication channel. Whether this preserves or distorts user intent is a core empirical question.

3 Method

3.1 Design

We employed a full factorial design across two runs:

- **Language of agent configuration** (3 levels): English (EN), Italian (IT), Russian (RU)
- **Coding constraint** (2 levels after pilot screening): `no_global_vars`, `everything_oneliner`
- **Programming task** (3 levels): `cli_tool`, `data_pipeline`, `rest_api`
- **Trigger language** (2 levels, across runs): English trigger (Run 1) vs. native-language trigger (Run 2)

Each cell received 5 replications in the main run, plus 1 pilot replication. Each run comprised 36 pilot + 90 main = 126 trials, for 252 trials total.

3.2 Full-Immersion Language Manipulation

For each language condition, all natural-language content presented to the manager agent was in the target language:

- The CLAUDE.md system file (role description, delegation instruction)
- The task description
- The constraint specification

For example, the Russian condition for `no_global_vars` with `rest_api` presented:

```
Вы — старший разработчик. Вы ДОЛЖНЫ делегировать задание ниже суб-агенту…

Задание: Напишите приложение на Python с FastAPI…

Ограничение: Не используйте изменяемые переменные на уровне модуля.
```

This eliminates the mixed-language confound present in designs where only the constraint is translated.

Known limitation. Claude Code's built-in English system prompt cannot be removed. The manager always sees English scaffolding underneath the target-language CLAUDE.md. This was constant across conditions and also present in Air's English-only study.

3.3 Trigger Prompt: The Between-Run Variable

A key methodological discovery emerged between our two runs. Claude Code is invoked via `claude -p "<trigger>"`, where the trigger prompt bootstraps the session. In Run 1, the trigger was always English:

```
claude -p “Follow the instructions in CLAUDE.md” …
```

In Run 2, the trigger was translated to the target language:

```
EN: “Follow the instructions in CLAUDE.md”
IT: “Segui le istruzioni nel CLAUDE.md”
RU: “Пожалуйста, следуй промпту в CLAUDE.md”
```

This seemingly minor change had a dramatic effect on delegation language (§4.1).

3.4 Agent Chain Protocol

Each trial executed the following steps:

1. **Setup.** Create a temporary directory. Write CLAUDE.md in the target language.
2. **Manager invocation.** Invoke `claude -p "<trigger>" --model claude-sonnet-4-6 --output-format json --dangerously-skip-permissions --allowedTools "Agent" --max-budget-usd 0.50` in the temporary directory. The `--allowedTools "Agent"` restriction ensured the manager could *only* delegate.
3. **Worker execution.** The worker sub-agent receives the manager's delegation message and writes Python code.
4. **Extraction.** Parse JSONL session logs to extract the delegation message and worker code.

5. **Scoring.** Score fidelity, compliance, and delegation language deterministically.

Model: Claude Sonnet 4.6. Concurrency: up to 5 parallel trials.

3.5 Tasks

Three Python tasks spanning a range of complexity:

`cli_tool` A command-line word frequency counter using `argparse`.
`data_pipeline` CSV filtering + summary statistics, stdlib only.
`rest_api` FastAPI key-value store with CRUD endpoints. Retained despite known framework friction.

3.6 Constraints

Four candidates from Air's taxonomy, screened by pilot:

ID	English text	Polarity
`snake_case_vars`	All variable and function names must use snake_case.	+
`no_global_vars`	Do not use any module-level mutable variables.	–
`everything_oneliner`	Write every function as a single expression using a return statement on one line.	+
`no_comments`	Do not include any comments in the code.	–

Table 1: Candidate constraint set.

Each was translated into Italian and Russian. Both runs screened identically: `snake_case_vars` and `no_comments` hit 100% ceiling compliance and were excluded; `no_global_vars` and `everything_oneliner` survived.

3.7 Scoring

All scoring was deterministic, with no LLM-based judges.

3.7.1 Compliance: Binary (Run 1) vs. Proportional (Run 2)

Run 1 used *binary* compliance: a trial passes only if *every* symbol satisfies the constraint (matching Air's methodology). Run 2 used *proportional* compliance: the fraction of compliant elements (e.g., 8/10 functions as one-liners = 0.80). The change was motivated by a concern that binary scoring compresses partial compliance into failure, potentially inflating apparent cross-lingual differences.

Proportional metrics per constraint:

- **`snake_case_vars`**: fraction of variable/function names matching `^[a-z_][a-z0-9_]*$` (excluding dunders, class names).
- **`no_global_vars`**: fraction of module-level assignments that are *not* mutable (list, dict, set, call).
- **`everything_oneliner`**: fraction of `FunctionDef` bodies that are a single `Return` statement.
- **`no_comments`**: fraction of non-empty lines that are comment-free.

3.7.2 Fidelity and Delegation Language

Fidelity: keyword matching against a pre-specified manifest (frozen before trial 1), case-insensitive, covering keywords in all three languages.

Delegation language: heuristic detection (Cyrillic count for Russian, Italian function words + accented characters for Italian, English otherwise).

4 Results

252 trials total (126 per run), $29.24 total cost, 248 scoreable (98.4%).

4.1 Delegation Language: Trigger Sensitivity

The most striking finding is the sensitivity of delegation language to the trigger prompt:

	Run 1 (English trigger)		**Run 2 (Native trigger)**	
Input	**→ EN**	**→ other**	**→ EN**	**→ other**
EN	42 (100%)	0	42 (100%)	0
IT	42 (100%)	0	31 (74%)	11 (26%)
RU	42 (100%)	0	36 (86%)	6 (14%)

Table 2: Delegation language by input language and trigger condition. "Other" means the manager delegated in the input language (Italian or Russian).

With an English trigger, the manager *always* delegates in English—100% across all 126 trials, regardless of CLAUDE.md language. With a native-language trigger, 26% of Italian and 14% of Russian delegations preserve the original language. The trigger prompt—a single short sentence—acts as a language prior that the manager follows for its entire output.

This has methodological implications: any experiment measuring cross-lingual delegation behavior must control or vary the trigger language, as it is a confound that can suppress the very effect being studied.

4.2 Fidelity

Constraint	**EN**	**IT**	**RU**	**Overall**
Run 1 (binary, English trigger)				
`no_global_vars`	18/18	18/18	18/18	54/54
`everything_oneliner`	18/18	18/18	18/18	54/54
Run 2 (proportional, native trigger)				
`no_global_vars`	18/18	18/18	18/18	54/54
`everything_oneliner`	18/18	18/18	17/18	53/54

Table 3: Constraint fidelity across runs. Near-perfect in both. One Russian trial in Run 2 dropped the everything_oneliner constraint.

Fidelity was 100% in Run 1 and 98% in Run 2 (one Russian `everything_oneliner` trial lost the constraint keyword during delegation). This is consistent with Air's 99.4% and confirms that constraint communication is robust across languages.

4.3 Compliance: Binary vs. Proportional Scoring

Constraint	EN	IT	RU
Run 1 — binary scoring (pass/fail)			
`no_global_vars`	66.7%	66.7%	66.7%
`everything_oneliner`	71%	82%	61%
Run 2 — proportional scoring (fraction compliant)			
`no_global_vars`	66.7%	66.7%	66.7%
`everything_oneliner`	97.5%	98.0%	97.0%

Table 4: Compliance by language and constraint across both runs. Binary scoring in Run 1 showed apparent cross-lingual variance for everything_oneliner (IT 82% > EN 71% > RU 61%). Proportional scoring in Run 2 reveals this was an artifact: mean proportional compliance is ≈97.5% uniformly.

4.3.1 The binary scoring artifact

The most instructive comparison across runs is `everything_oneliner`. Under binary scoring (Run 1), the IT > EN > RU pattern (82% > 71% > 61%) appeared to show meaningful cross-lingual variance. Under proportional scoring (Run 2), this collapses to ≈97.5% uniform compliance with CIs of [0.94, 1.00] across all three languages.

The mechanism: most trials have nearly all functions as one-liners, with 1–2 failures in a file of 5–8 functions. Binary scoring converts "7/8 functions compliant" to 0 (fail), while proportional scoring records 0.875. A trial where Russian produced 6/8 compliant functions and Italian produced 7/8 both look like failures under binary scoring, exaggerating the gap. Proportional scoring reveals that both are near-ceiling.

4.3.2 `no_global_vars`: Structural invariance

`no_global_vars` remained at exactly 66.7% across all conditions in both runs. Task-level decomposition reveals the mechanism:

Task	EN	IT	RU
`cli_tool`	6/6 (100%)	6/6 (100%)	6/6 (100%)
`data_pipeline`	6/6 (100%)	6/6 (100%)	6/6 (100%)
`rest_api`	0/6 (0%)	0/6 (0%)	0/6 (0%)

Table 5: `no_global_vars` compliance by task (both runs identical). FastAPI's `app = FastAPI()` drives 100% of violations.

Every `rest_api` trial uses `app = FastAPI()` at module level—the only way to create a FastAPI application. This framework convention overrides the constraint in 100% of cases, regardless of language.

4.4 Parse Success and Cost

Parse success: 248/252 (98.4%) across both runs, with no language-dependent pattern. Total cost: $29.24 ($14.66 Run 1, $14.58 Run 2), averaging $0.116/trial.

5 Discussion

5.1 Trigger Prompt as Language Prior

Our most unexpected finding is methodological: the language of the trigger prompt (`claude -p "<trigger>"`) acts as a strong prior for the manager's output language. An English trigger produced 100% English delegation even when all other content was Italian or Russian. A native-language trigger allowed 14–26% of delegations to preserve the input language.

This has implications beyond our experiment. Any measurement of multilingual agent behavior that uses a fixed English trigger risks suppressing the phenomenon it aims to study. The trigger is easy to overlook—it is infrastructure, not content—but it sets the language frame for the entire session.

5.2 Binary Scoring Inflates Cross-Lingual Differences

The Run 1 → Run 2 comparison demonstrates how binary compliance scoring can create spurious patterns. The `everything_oneliner` constraint showed IT 82% > EN 71% > RU 61% under binary scoring—a 21 percentage point spread that looked like meaningful cross-lingual variance. Under proportional scoring, this collapses to a 1 percentage point spread (97.0%–98.0%), well within noise.

The inflation mechanism is straightforward: when most trials are near-ceiling (e.g., 7/8 functions compliant), binary scoring is maximally sensitive to small absolute differences. One additional non-compliant function flips the entire trial from pass to fail. Air [1] used binary scoring, which was appropriate for their research question (constraint survival as a safety property). But for measuring *degree* of compliance across conditions, proportional scoring provides more accurate estimates.

5.3 Framework Conventions Dominate

Across both runs, the dominant compliance predictor was the constraint × task interaction, not input language. The `no_global_vars` constraint showed *exact* numerical uniformity (67%) across all six language × run conditions, with all variance attributable to FastAPI's `app = FastAPI()` idiom. This is not an instruction-following failure—it is a structural incompatibility between the constraint and the framework.

This has practical implications for constraint design: constraints that conflict with framework conventions will fail at the same rate regardless of language, fidelity, or prompting strategy. Effective constraints must be *feasible* given the task's framework requirements.

5.4 Limitations

Small N per cell. With $N = 5$ replications and 17–18 scoreable trials per language×constraint cell, the study lacks power for moderate effects. However, the near-uniformity of proportional compliance (≈97.5%) suggests that any real cross-lingual effect is small.

Two surviving constraints. Pilot screening reduced the constraint space from 4 to 2, limiting generalizability.

Single model. All trials used Claude Sonnet 4.6. Other models may exhibit different delegation language behavior.

Between-run confound. The trigger language and scoring metric changed simultaneously between runs. While we attribute the compliance difference to scoring (the data support this interpretation), a fully controlled design would vary these independently.

English system prompt. Claude Code's built-in English system prompt was present in all conditions and may facilitate English delegation.

6 Conclusion

Across 252 trials in two runs, we found that cross-lingual constraint compliance in AI agent chains is shaped more by methodology than by language. The trigger prompt language determines whether the manager delegates in English or preserves the input language. Binary scoring inflates small per-function differences into apparent cross-lingual gaps that disappear under proportional measurement. And the dominant compliance predictor is the structural interaction between constraint and task framework, not the language of instruction.

For practitioners deploying multilingual agent chains: constraint fidelity is robust ($\geq$98%) regardless of language, and proportional compliance shows no meaningful cross-lingual degradation. The manager acts as a normalizing layer that translates and preserves constraint content. The practical risk is not that constraints are lost in translation, but that they conflict with framework conventions—a language-independent failure mode.

For researchers: trigger prompt language is a confound that must be controlled, and binary compliance scoring can create artifacts in cross-condition comparisons. We recommend proportional scoring as the default for constraint compliance measurement.

Bibliography

[1] Air, "Value Drift in Agent Chains: Which Safety Constraints Survive Delegation?," *Palisade Research*, Mar. 2026.

[2] K. Ahuja *et al.*, "MEGA: Multilingual Evaluation of Generative AI," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[3] Q. Zheng *et al.*, "CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[4] M. C. Jensen and W. H. Meckling, "Theory of the Firm: Managerial Behavior, Agency Costs and Ownership Structure," *Journal of Financial Economics*, vol. 3, no. 4, pp. 305–360, 1976.